

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1205	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1-ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	M.harini	Reg.No.:	192424011
Department:	AI&DS	Date:	

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I M.Harini (192424011) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1. Addition and Subtraction Using 2's Complement (8-bit)

Given

- Number A = +45
- Number B = -23
- Representation: 8-bit 2's complement

Step 1: Convert to Binary

+45

45 = 00101101

-23

23 = 00010111

Take 2's complement:

- 1's complement = 11101000
- Add 1 = 11101001

Therefore,

-23 = 11101001

(A) Addition (+45 + (-23))

00101101 (+45)
+ 11101001 (-23)

1 00010110

Ignore carry out.

Result:

00010110 = +22

Decimal Check

45 + (-23) = 22

Correct.

Overflow Check

Overflow occurs only when

- Positive + Positive = Negative
- Negative + Negative = Positive

Here,

Positive + Negative = Positive

No Overflow

(B) Subtraction (+45 - (-23))

Subtracting a negative means adding its positive.

45 - (-23)

= 45 + 23

23 in binary

00010111

Now add

00101101
+ 00010111

01000100

Result

01000100 = 68

Decimal Check

$$45 - (-23) = 68$$

Correct.

Sign Bit Alignment

Since both numbers are already stored in **8 bits**, their sign bits are automatically aligned.

+45

00101101

Sign bit = 0

-23

11101001

Sign bit = 1

No extra alignment is required because both operands have the same bit width.

Overflow Detection

The processor detects overflow using either method.

Method 1

Compare carry into MSB and carry out of MSB.

If different

→ Overflow

If same

→ No overflow

Method 2

Using sign bits

Operands Result Overflow

+	+	-	Yes
---	---	---	-----

-	-	+	Yes
---	---	---	-----

+	-	Any	No
---	---	-----	----

Final Results

Addition

$$45 + (-23) = 22$$

Overflow = **No**

Subtraction

$$45 - (-23) = 68$$

Overflow = **No**

2. Carry Look-Ahead Adder (CLA)

Suppose

$$A = 1010$$

$$B = 0111$$

$$C_{in} = 0$$

Step 1: Generate (G) and Propagate (P)

Formulas

$$G_i = A_i \times B_i$$

$$P_i = A_i \oplus B_i$$

Bit A B G P

0 0 1 0 1

1 1 1 1 0

2 0 1 0 1

3 1 0 0 1

Step 2: Carry Equations

$$C_1 = G_0 + P_0 C_{in}$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_{in}$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{in}$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{in}$$

All carries are calculated **simultaneously**.

Ripple Carry Adder

Carry must move

Bit0 $\rightarrow$ Bit1 $\rightarrow$ Bit2 $\rightarrow$ Bit3

Each stage waits for the previous carry.

Delay

$$4 \times \text{Carry Delay}$$

Carry Look-Ahead Adder

All carries are computed together.

Delay

Generate + Propagate + Sum

$\approx$ Constant time

Comparison

Ripple Carry

Carry travels bit by bit

Slow

Simple hardware

Higher delay

Carry Look-Ahead

Carries generated simultaneously

Fast

More hardware

Lower delay

Why CLA is Faster

- No waiting for previous carry.
- Parallel carry computation.
- Reduces propagation delay.
- Used in modern CPUs and ALUs.

3. Booth's Algorithm ($-6 \times +5$)

Step 1

Multiplicand (M)

-6

0110

2's complement

1010

Multiplier (Q)

+5

0101

Accumulator

A = 0000

Booth Bit

Q-1 = 0

Initial

A = 0000

Q = 0101

Q-1 = 0

Rule

Q0 Q-1 Operation

00 No operation

11 No operation

01 $A = A + M$

10 $A = A - M$

After every operation

Arithmetic Right Shift

Cycle 1

Q0Q-1

10

Subtract M

Shift

Cycle 2

Pattern

11

No operation

Shift

Cycle 3

Pattern

01

Add M

Shift

Cycle 4

Pattern

10

Subtract M

Shift

Final Result

11100010

Decimal

-30

Correct because

$$-6 \times 5 = -30$$

Why Booth's Algorithm is Efficient

- Works directly with signed numbers.
- Consecutive 1's are handled together.
- Fewer additions/subtractions.
- Faster than ordinary shift-and-add multiplication.

Registers Used

Accumulator (A)

Stores partial product.

Multiplier (Q)

Contains multiplier bits.

Multiplicand (M)

Number being multiplied.

Booth Bit (Q-1)

Determines the operation.

4. Division ($13 \div 3$)

Dividend

13

1101

Divisor

3

0011

Expected Answer

Quotient =4

Remainder =1

(A) Restoring Division

Steps

1. Shift left AQ
2. Subtract divisor
3. If result negative

Restore remainder

Quotient bit =0

Otherwise
Keep result
Quotient bit =1
Repeat.

Final

Quotient =0100

Remainder =0001

(B) Non-Restoring Division

Steps

- Shift left
- If previous remainder positive

Subtract divisor

- If previous remainder negative

Add divisor

No immediate restoring.

Only one correction at the end if remainder is negative.

Final

Quotient =0100

Remainder =0001

Comparison

Restoring

Restores negative remainder

More operations

Higher execution time

Simpler concept

Non-Restoring

No restoration after every step

Fewer operations

Lower execution time

More efficient

Why Non-Restoring is Preferred

- Eliminates unnecessary restoration.
- Fewer arithmetic operations.
- Faster hardware implementation.
- Widely used in modern processors.

5. IEEE 754 Floating Point Representation

Given

-13.25

Step 1: Convert to Binary

Integer

13

1101

Fraction

0.25

0.01

Binary

1101.01

Normalized

1.10101×2^3

Single Precision (32-bit)

Sign

Negative

1

Exponent

Bias

127

Actual exponent

3

Stored exponent

130

10000010

Mantissa

Remove leading 1

101010000000000000000000

Final

1

10000010

101010000000000000000000

Double Precision (64-bit)

Sign

1

Exponent

Bias

1023

$1023+3$

1026

10000000010

Mantissa

1010100

Floating Point Addition

Example

13.25

+

5.5

Step 1

Compare exponents.

Step 2

Shift smaller mantissa.

Step 3

Add mantissas.

Step 4

Normalize result.

Step 5

Round according to IEEE 754 rules.

Precision Issues

Normalization

Moves binary point so that the number is in the form:

$$1.xxxxx \times 2^n$$

Rounding

Extra bits are rounded using IEEE 754 rounding rules (commonly **Round to Nearest, Ties to Even**).

Precision Loss

Occurs because:

- Mantissa has limited bits.
- Some decimal fractions (e.g., 0.1) cannot be represented exactly in binary.
- Small values may be lost when added to very large values due to exponent alignment.

Comparison

Feature	Single Precision	Double Precision
Total Bits	32	64
Sign Bits	1	1
Exponent Bits	8	11
Mantissa Bits	23	52
Precision	≈7 decimal digits	≈15–16 decimal digits
Memory	4 Bytes	8 Bytes

These answers cover the binary conversions, intermediate steps, algorithm operations, comparisons, and explanations typically expected for 10–15 mark university COA questions.